

CS6650 HW6

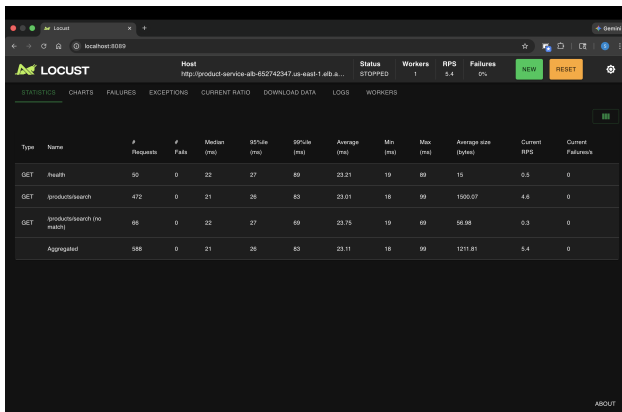
Sai Karthikeyan Sura
February 22nd, 2026

Part II: Identifying Performance Bottlenecks

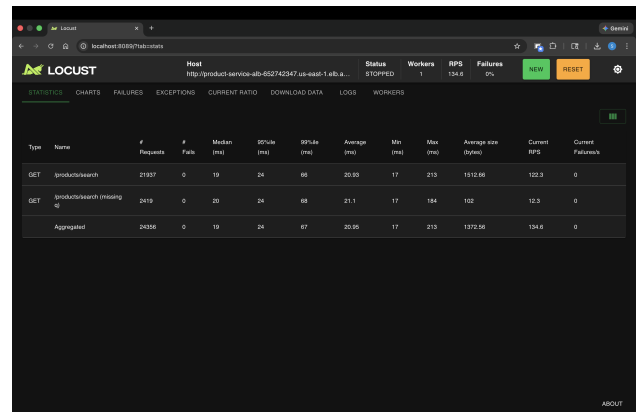
The product search service stores 100,000 products in a `sync.Map` at startup. Each search request checks 100 products starting at a random offset, performs case-insensitive matching on name and category fields, and returns at most 20 results. The service was deployed on ECS Fargate with 256 CPU units (0.25 vCPU) and 512 MB memory.

Test	Users	Requests	RPS	Median	p95	p99	Max
Baseline (256 CPU)	5	588	5.4	21ms	26ms	83ms	99ms
Breaking Point (256 CPU)	20	24,356	134.6	19ms	24ms	67ms	213ms
Breaking Point (512 CPU)	20	24,247	135.3	20ms	34ms	58ms	262ms

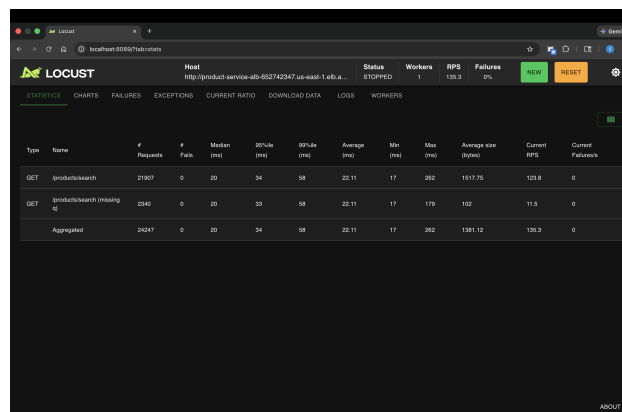
Table 1: Load test statistics across single-instance configurations.



(a) Baseline (256 CPU): 5 users



(b) Breaking Point (256 CPU): 20 users



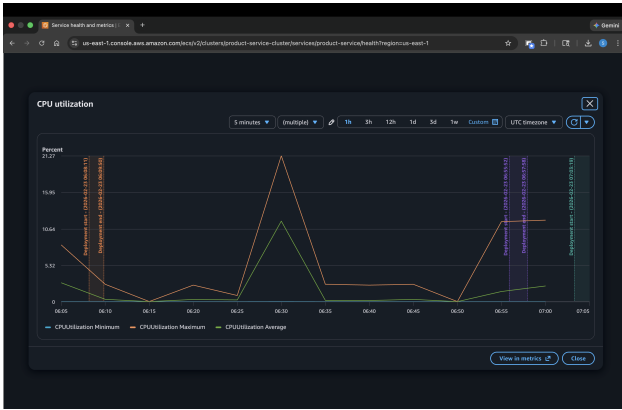
(c) Breaking Point (512 CPU): 20 users

Figure 1: Single-instance load test comparison baseline, 256-CPU and 512-CPU breaking points.

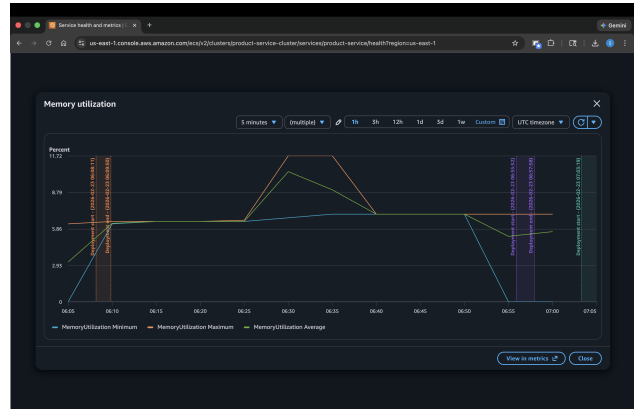
CloudWatch Metrics Analysis

Figure 2 shows CPU and memory utilization across sequential load tests. The baseline test with 5 users on a 256 CPU instance produced moderate CPU usage, while the 20-user breaking

point test briefly saturated the instance at 100% CPU. Repeating the breaking point test on a 512 CPU instance significantly reduced peak utilization, illustrating the effect of vertical scaling.



(a) CPU utilization



(b) Memory utilization

Figure 2: CloudWatch resource utilization.

Which resource hits the limit first? CPU is the limiting resource. The 20-user breaking point test on a 256 CPU instance produced brief spikes to 100% CPU. Memory remained stable, indicating the workload is not memory-bound.

How much did response times degrade? Despite CPU saturation, median response times remained low (19–21ms), and the service achieved 134.6 RPS. P95 and P99 latencies increased modestly (24–67ms), showing that brief CPU saturation has limited impact on overall latency for this workload.

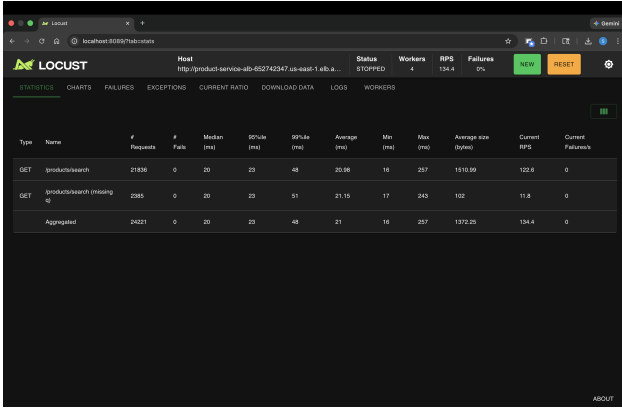
Could you solve this by doubling CPU (256 → 512)? Increasing to 512 CPU reduced peak CPU utilization during the same 20-user load, confirming that vertical scaling alleviates compute pressure. RPS increased slightly to 135.3, and p99 dropped to 58ms. However, because individual requests are computationally inexpensive, vertical scaling yields limited performance gains. Sustained throughput improvements require horizontal scaling to distribute load across multiple instances.

Part III: Horizontal Scaling with Auto Scaling

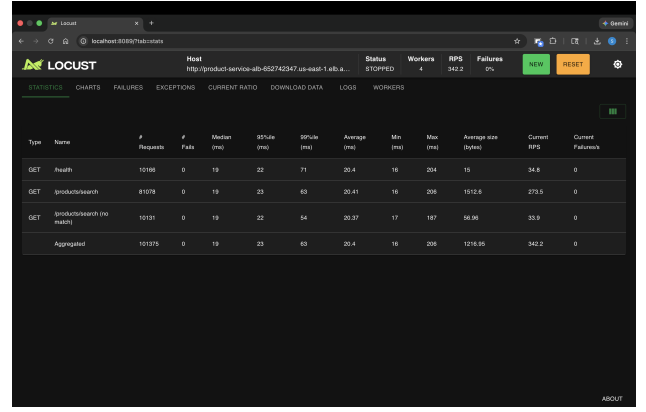
The service was redeployed with an Application Load Balancer (ALB) distributing traffic across ECS Fargate tasks, with auto scaling configured to maintain 70% average CPU utilization (min 2, max 4 instances). Compared to the single-instance results in Table 1, horizontal scaling enables significantly higher aggregate throughput while maintaining low latency.

Test	Users	Requests	RPS	Median	p95	p99	Max
Breaking (2 inst)	20	24,221	134.4	20ms	23ms	48ms	257ms
Stress (2–4 inst)	50	101,375	342.2	19ms	23ms	63ms	206ms
Spike (2–4 inst)	50	115,510	644.6	20ms	25ms	85ms	398ms
100 Users (2–4 inst)	100	225,709	1269.6	19ms	46ms	120ms	316ms
Resilience (1 task down)	20	68,468	134.9	20ms	24ms	37ms	200ms
Resilience (2 tasks down)	50	62,666	339.9	19ms	22ms	62ms	205ms
50% CPU Target	20	101,344	333.3	20ms	25ms	78ms	287ms

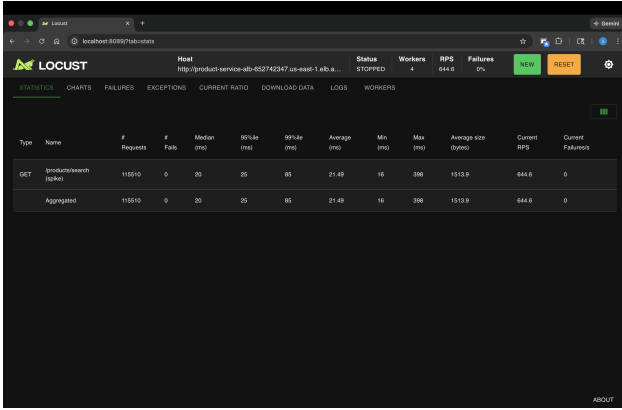
Table 2: Horizontal scaling load test statistics.



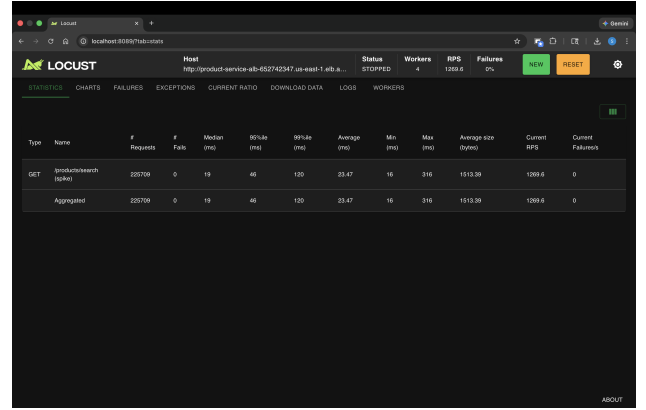
(a) Breaking (2 instances): 20 users



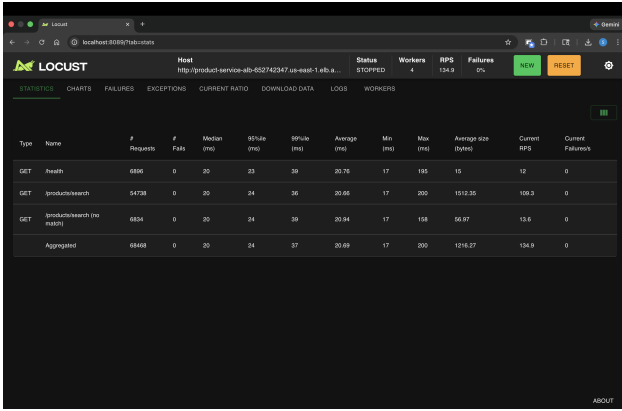
(b) Stress (2-4 instances): 50 users



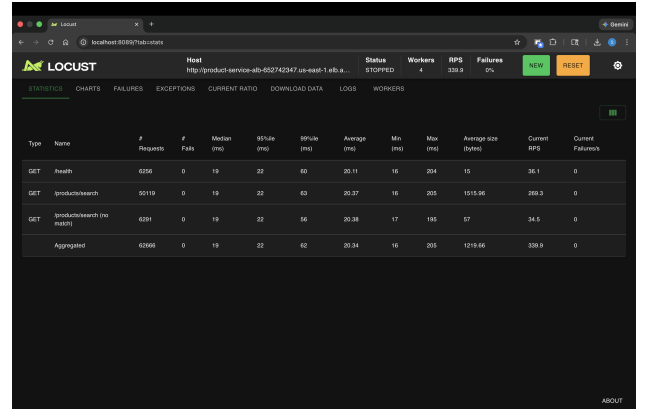
(c) Spike (2-4 instances): 50 users



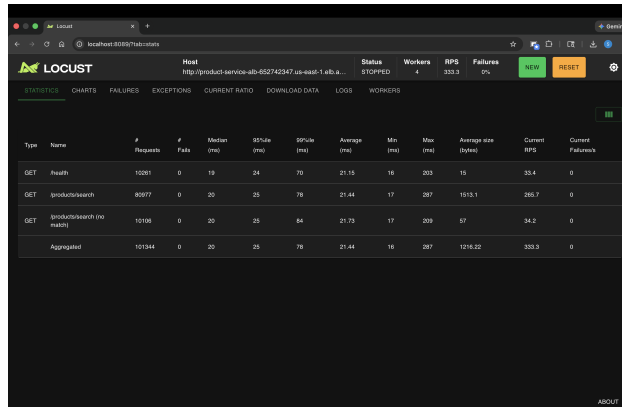
(d) Extreme Spike (2-4 instances): 100 users



(e) Resilience (1 task down): 20 users



(f) Resilience (2 tasks down): 50 users

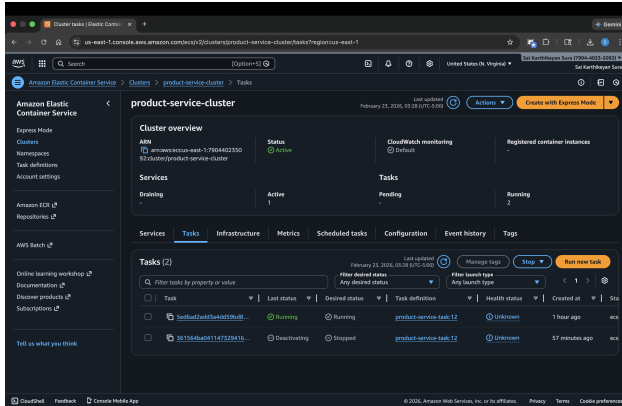


(g) 50% CPU Target Tracking: 20 users

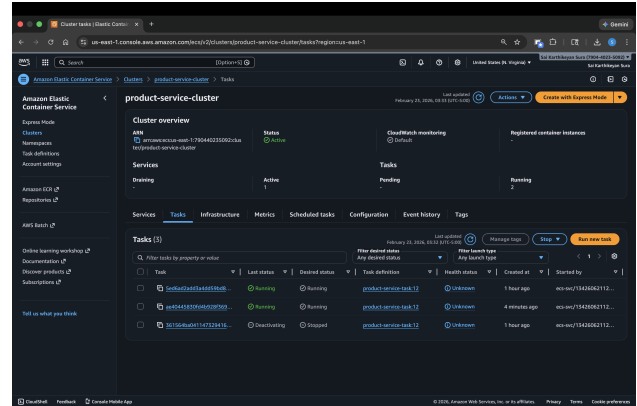
Figure 3: Horizontal scaling experiments.

Resilience Testing

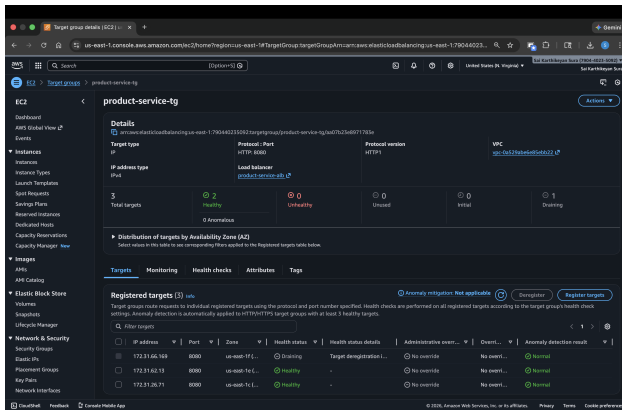
During load testing, one ECS task was manually stopped; the ALB drained connections and ECS launched a replacement, resulting in 0% request failures.



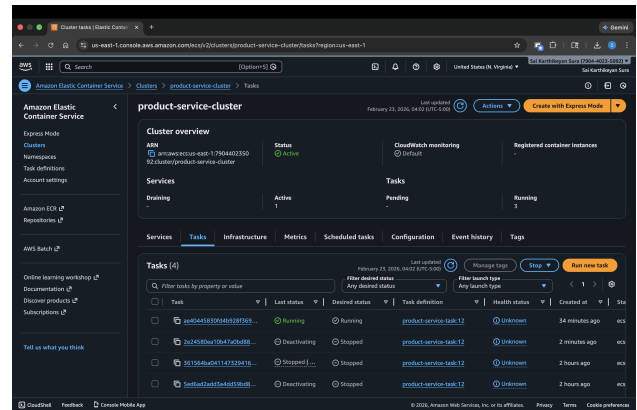
(a) Manual deactivation of 1 task (20 users)



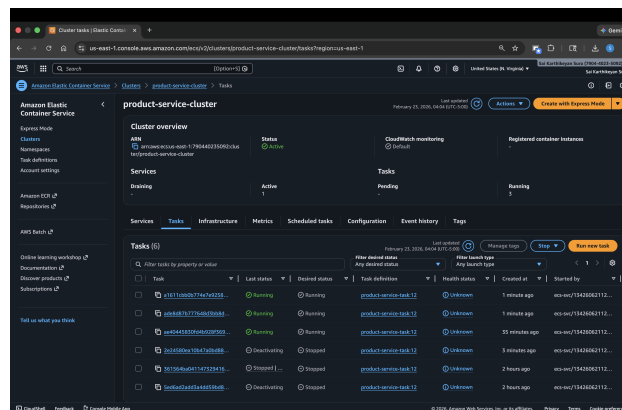
(b) Auto replacement of 1 task



(c) Target group draining during shutdown



(d) Manual deactivation of 2 tasks (50 users)



(e) Auto replacement of 2 tasks under higher load

Figure 4: Seamless task draining and automatic replacement during failures.

Component Roles

Application Load Balancer (ALB): Receives HTTP traffic on port 80 and distributes requests across healthy Fargate tasks via round-robin, providing a stable DNS endpoint independent of task count.

Target Group: Performs health checks on `/health` every 30 seconds; tasks failing 3 consecutive checks are deregistered and replaced to ensure traffic is routed only to healthy instances.

Auto Scaling: Maintains 70% average CPU utilization by scaling task count up or down, with a 300-second cooldown for scale-in events.

Monitoring

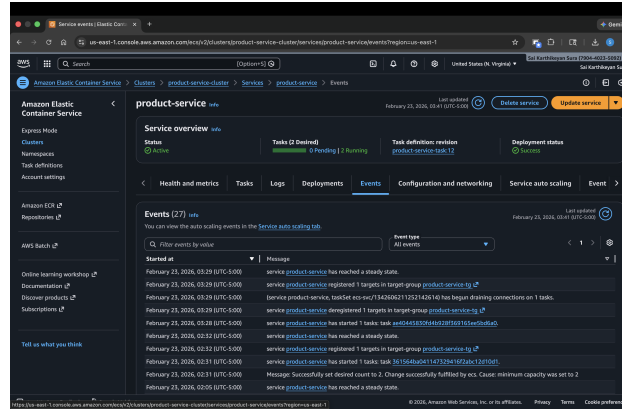
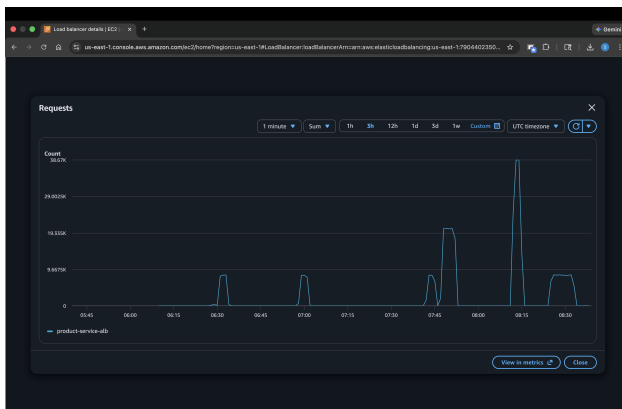
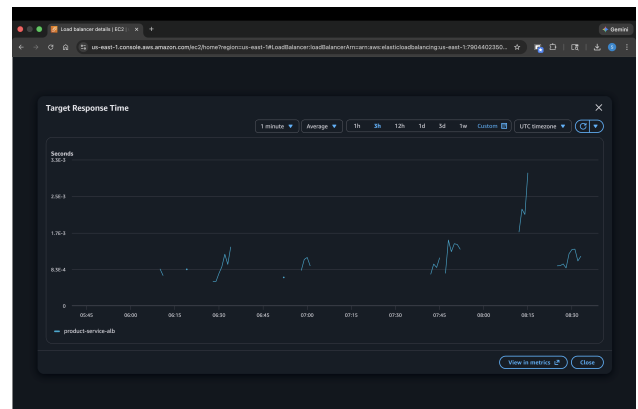


Figure 5: ECS Events log of auto scaling, task registration/deregistration and count changes.



(a) ALB request count per minute



(b) ALB target response time

Figure 6: ALB metrics show request volume scaling with users while latency stays below 3 ms.

Vertical vs. Horizontal Scaling

Dimension	Vertical Scaling	Horizontal Scaling
Approach	Increase CPU/memory per instance	Add more instances
Complexity	Update task definition	Requires ALB, target groups, policies
Fault tolerance	Single point of failure	Survives individual instance failures
Cost curve	Diminishing returns	Near-linear cost scaling
Result	256→512 CPU: no improvement	1→2+ instances: 2.5× RPS at 50 users

Table 3: Vertical vs. horizontal scaling trade-offs.

Scaling Behavior Predictions

System throughput scales near-linearly with user count: 2–4 instances handled 50 users at 342–644 RPS and 100 users at 1,269 RPS with 0% failures and modest p95 increase (46 ms vs. 23 ms). A 50% CPU target triggers earlier scaling, improving responsiveness at moderate load, while 70% is more cost-efficient for steady workloads.